Deep Learning - EX1

noam azulay 213705957
ofri tabib 203120258

April 18, 2026

Antigen Discovery for SARS-CoV-2

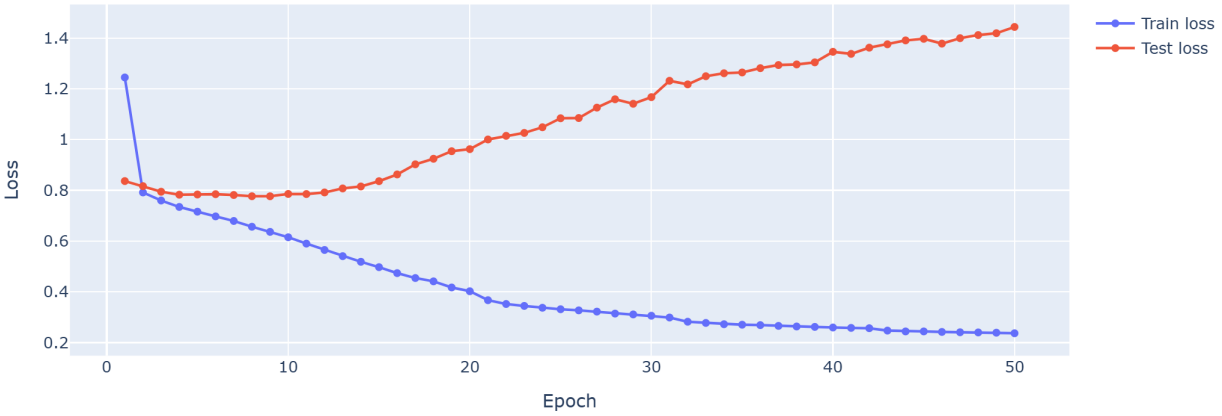
Model input and output

Each 9-mer is encoded as a flat 180-dim vector (9 AAs × 20 values each).
instead of one hot encoding we define a matrix that show quantify the biological distance between amino acids.
basically each row represent an amino acids and similarity to other AAs instead of assuming all AAs are dissimilar.

The model outputs 7 logits (6 MHC alleles + 1 negative class), converted to probabilities via softmax.
Each label is one-hot encoded. CrossEntropyLoss is used since this is single-label multi-class classification — a peptide binds to at most one allele. At inference, argmax gives the predicted class; '1 − P(negative)' gives overall binding confidence.

Simple MLP with hidden layers

Input dimension: 180 (9 positions × 20 BLOSUM62 values each).
MLP architecture: 180 → 180 → 180 → 7 (two hidden layers matching input dim, ReLU activations, no dropout, no augmentation).
Results:



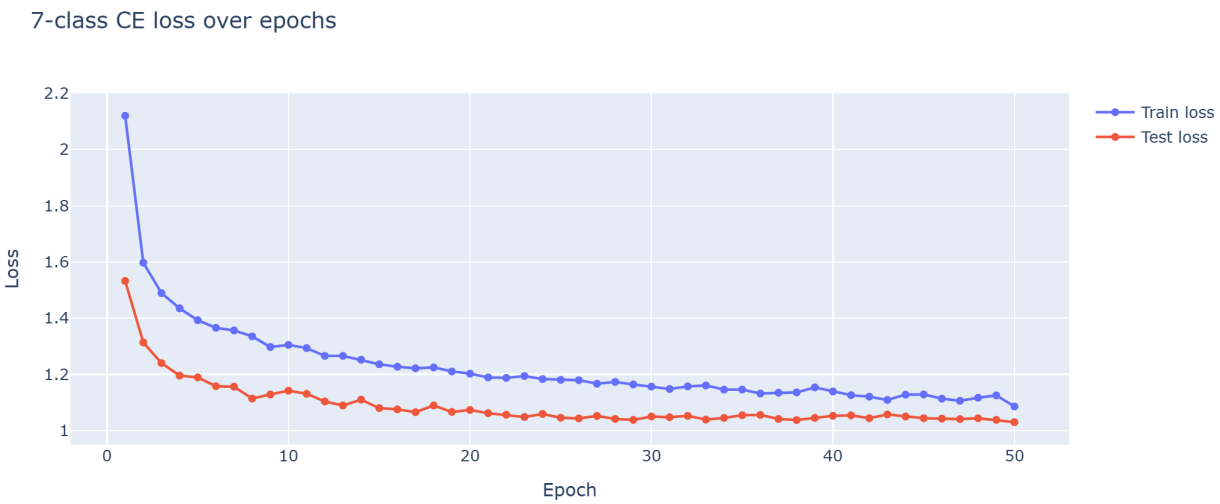
This setup shows severe overfitting, the train loss drops steadily while the test loss diverges upward after ~10 epochs.
The 180-dim hidden layers give ~32k parameters per layer (180×180), which is large relative to the dataset. Without dropout

or augmentation the model memorizes training sequences rather than generalizing. The dimension itself is not problematic on its own, but keeping it constant across all layers (no bottleneck, no compression) removes the inductive bias that forces the network to learn compact representations.

Network Architecture

Layer stack: 180 → 128 → 64 → 32 → 16 → 7
A funnel (compression) design — each hidden layer halves the dimension. This forces the network to learn increasingly compact representations, acting as an inductive bias against memorization.
Each hidden layer: Linear → ReLU → Dropout(0.3) Output layer: Linear(16 → 7), no activation (raw logits for CrossEntropyLoss).
Other design choices:

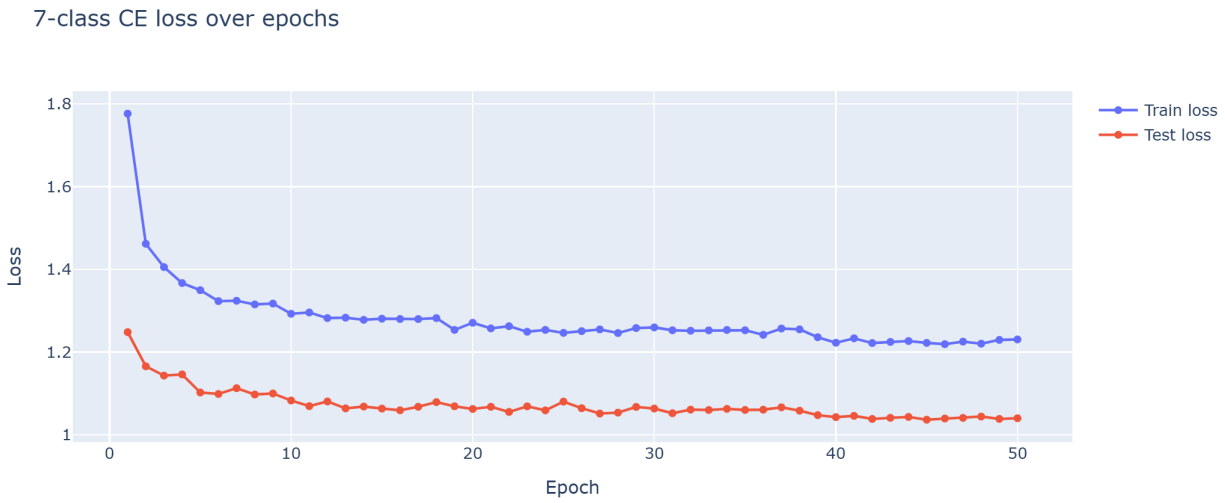
- Dropout 0.3 on every hidden layer — regularization against overfitting
- Class-weighted loss with neg_multiplier=3.0 — upweights the negative class (majority) to prevent the model from ignoring positives
- ReduceLROnPlateau scheduler — halves LR when test loss plateaus, patience=10
- BLOSUM62 augmentation (baseline.yaml) — random single-AA substitution during training;



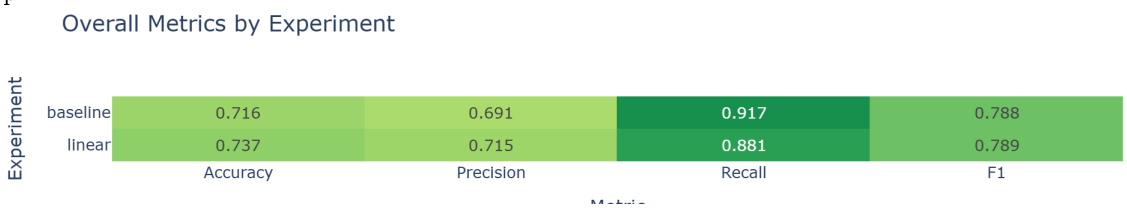
Overall accuracy: 72% | F1: 0.787
Train and test losses decrease together — no divergence, much healthier than same hidden layers.
The funnel compresses 180 dims down to 16 before classification, reducing parameters per layer and encouraging generalization.

Linear model

Same architecture (180 → 128 → 64 → 32 → 16 → 7) but ‘act_fn = None‘ — stacked linear layers with no nonlinearity.
Results:



comparison to baseline:

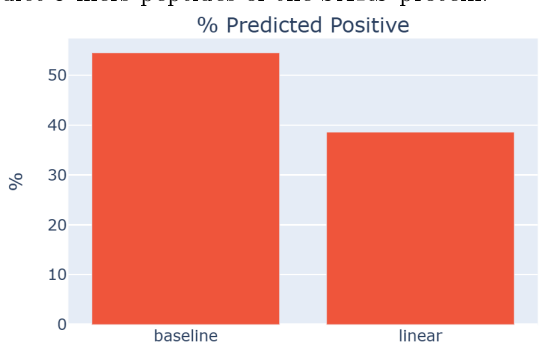


Stacked linear layers collapse to a single linear map — the hidden layers add no expressive power. The near-identical F1 reveals that MHC-peptide binding is largely a linear problem: binding is dominated by a few anchor positions (typically positions 2 and 9 in a 9-mer) with strong amino acid preferences, which are position-specific additive signals that a linear model captures well.

The ReLU network's advantage is in recall — its nonlinearities let it learn interactions between positions, picking up weaker or allele-specific binding patterns the linear model misses. The cost is more false positives, as some learned nonlinear combinations don't generalize. For this task, the marginal gain from nonlinearity is small because the dominant signal is positional and additive.

SARS prediction

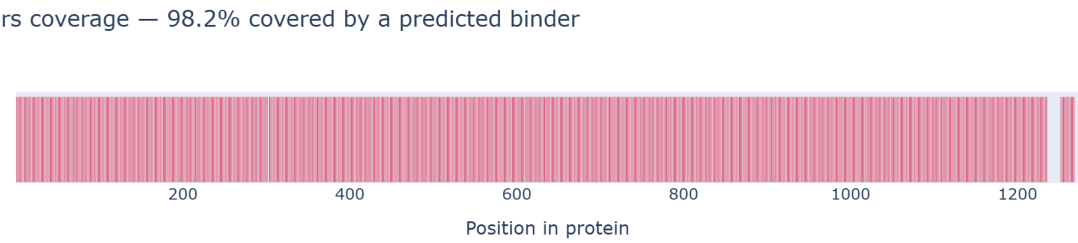
predict 9-mers peptides of the SARS protein:



Top peptides (baseline):

rank	peptide	allele	confidence	position
1	LNDILSRLD	A0101	0.9942	977
2	ESEFRVYSS	A0101	0.9934	154
3	STEIYQAGS	A0101	0.9831	469

Protein coverage (baseline):



Theoretical Questions

1. Composition of linear functions:
   let $f, g : V \to V$ be linear functions, let h be $h(x) = f(g(x))$ then for $x, y \in V$, and scalar a :

$$\begin{aligned} h(ax+y) &= f(g(ax+y)) \\ &= f(ag(x) + g(y)) && (g \text{ is linear}) \\ &= af(g(x)) + f(g(y)) && (f \text{ is linear}) \\ &= ah(x) + h(y) && (h \text{ definition}) \end{aligned}$$

   hence h the composition of linear functions is also linear ■
   let f, g be $f(x) = Ax + b$,$g(x) = Cx + d$ where A, C are linear matrices and b, d are vectors, then:

$$\begin{aligned} f \circ g(x) = f(g(x)) =& f(Cx+d) \\ =& A(Cx+d) + b \\ =& (AC)x + (Ad+b) \end{aligned}$$

   therefore $f \circ g$ is an affine transformation ■

2. Prediction loss for circular angle
   $L = 1 - \cos(\theta_{pred} - \theta_{target})$
   since $\cos(-\theta) = \cos(\theta)$ guarantee that sign of the difference is ignored.
   when $\theta_{pred} = \theta_{target} \Rightarrow \cos 0 = 1 \Rightarrow L = 0$
   for the worst case: $\theta_{pred} - \theta_{target} = 180$ than $\cos 180 = -1 \Rightarrow L = 2$
   loss function is a linear combination of differentiable functions hence also differentiable and suitable for ML model that uses gradients.

3. Chain rule

(a)

$$\begin{aligned}\frac{\partial}{\partial x}f\left(x+y,2x,z\right)=&\frac{\partial}{\partial x}f_1^{'}\left(x+y,2x,z\right)+\frac{\partial}{\partial x}f_2^{'}\left(x+y,2x,z\right)+\frac{\partial}{\partial x}f_3^{'}\left(x+y,2x,z\right)\\=&f_1^{'}\left(x+y,2x,z\right)\frac{\partial\left(x+y\right)}{\partial x}+f_2^{'}\left(x+y,2x,z\right)\frac{\partial 2x}{\partial x}+f_3^{'}\left(x+y,2x,z\right)\frac{\partial z}{\partial x}\\=&f_1^{'}\left(x+y,2x,z\right)\cdot 1+f_2^{'}\left(x+y,2x,z\right)\cdot 2+f_3^{'}\left(x+y,2x,z\right)\cdot 0\\=&f_1^{'}\left(x+y,2x,z\right)+2f_2^{'}\left(x+y,2x,z\right)\end{aligned}$$

where $f_i^{'}$ is the partial derivatives of f with respect to argument number.

(b)

$$\begin{aligned}f_1\left(f_2\left(\ldots f_n\left(x\right)\right)\right)&=f_1^{'}\left(f_2\left(\ldots f_n\left(x\right)\right)\right)\cdot\left(f_2\left(\ldots f_n\left(x\right)\right)\right)^{'}\\&=\prod_{i=1}^{n-1}f_i^{'}\left(\ldots f_n\left(x\right)\right)\cdot f_n^{'}\left(x\right)\end{aligned}$$

(c) $f_1\left(x,f_2\left(x,f_3\left(\ldots f_{n-1}\left(x,f_n\left(x\right)\right)\right)\right)\right)$
Let

$$\begin{aligned}v_n&=f_n\left(x\right)\\v_{n-1}&=f_{n-1}\left(x,\right)v_n\\&\vdots\\v_2&=f_2\left(x,v_3\right)\\f_1\left(x,f_2\left(x,f_3\left(\ldots f_{n-1}\left(x,f_n\left(x\right)\right)\right)\right)\right)&=f_1\left(x,v_2\right)\end{aligned}$$

then:

$$\frac{d}{dx}f_1\left(x,v_2\right)=\partial_1 f_1\left(x,v_2\right)+\partial_2 f_1\left(x,v_2\right)\frac{dv_2}{dx}$$

note that for $1\leq i\leq n-1$:

$$\frac{dv_i}{dx}=\partial_1 f_i\left(x,v_{i+1}\right)+\partial_2 f_i\left(x,v_{i+1}\right)\frac{dv_{i+1}}{dx}$$

for $i=n$:

$$\frac{dv_n}{dx}=f_n^{'}\left(x\right)$$

by recursive:

$$\frac{d}{dx}f_1\left(x,v_2\right)=\partial_1 f_1+\partial_2 f_1\left(\partial_1 f_2+\partial_2 f_2\left(\ldots\partial_1 f_{n-1}+\partial_2 f_{n-1}\cdot f_n^{'}\left(x\right)\right)\right)$$

(d)

$$\begin{aligned}f\left(x+g\left(x+h\left(x\right)\right)\right)=&f^{'}\left(x+g\left(x+h\left(x\right)\right)\right)\cdot\left(x+g\left(x+h\left(x\right)\right)\right)^{'}\\=&f^{'}\left(x+g\left(x+h\left(x\right)\right)\right)\cdot\left(1+g^{'}\left(x+h\left(x\right)\right)\right)\\=&f^{'}\left(x+g\left(x+h\left(x\right)\right)\right)\cdot\left(1+g^{'}\left(x+h\left(x\right)\right)\cdot\left(x+h\left(x\right)\right)^{'}\right)\\=&f^{'}\left(x+g\left(x+h\left(x\right)\right)\right)\cdot\left(1+g^{'}\left(x+h\left(x\right)\right)\cdot\left(1+h^{'}\left(x\right)\right)\right)\end{aligned}$$